*2026*학년도 캡스톤 프로젝트

모찌 프로젝트 결과보고서

멀티모달 *AI* 기반 개인 기록 관리 서비스



MOCHI

흩어진 기록의 저장부터 해석과 정리, 검색과 대화, 필요한 순간의 재활용까지 연결한 iOS 서비스

모찌는 스크린샷·링크·메모를 하나의 기록 모델로 관리한다. 멀티모달 AI로 저장 의도를 해석하고, 기록을 근거로 답하며, 일정·할 일과 회수 알림으로 후속 행동을 돕는다.

구분	내용
프로젝트	모찌 MOCHI
팀	세얼간이
팀원	송준서 · 박세현 · 이용찬
학과	소프트웨어학과
학번	송준서 32202337 · 박세현 32211690 · 이용찬 32213336
지도교수	이성한 교수
제출일	2026년 11월 16일
수상	2026 SW중심대학 디지털 경진대회 SW부문 최우수상
저장소	https://github.com/dku-mochi/mochi

보고서 개요

모찌의 핵심은 정보를 저장하는 기능을 하나 더 만드는 데서 그치지 않고, 저장한 정보를 다시 찾고 행동으로 옮기는 흐름을 구현한 것이다. 이를 위해 분석·검색·회수 계획을 분리하고 실험 결과를 각 설계 선택에 반영하였다.

핵심 문제 해결

고민한 문제	비교와 선택	확인한 결과
무엇을 남긴 기록인가	추출 전략·모델·의도 구조와 분류 기준 개선	텍스트 40건의 비교와 229케이스 회귀 평가
어떻게 다시 찾을 것인가	임베딩·리랭커·반환 기록 수 비교	상위 순위 개선과 다정답 회수 범위 확대
언제 다시 알려줄 것인가	독립 계획과 맥락을 공유하는 계획 비교	실험 조건에서 비활동 시간대 배치·과잉 알림 감소

목차

장	내용	쪽
1	개발 배경과 요구사항	3
2	사용자 경험과 서비스 기능	4–5
3	시스템과 데이터 설계	6–8
4	핵심 기능 구현	9–12
5	문제 해결과 실험	13–18
6	검증 체계와 운영 관측	19–20
7	구현 성과와 향후 개선	21–22
8	팀과 프로젝트 산출물	23
9	참고자료	24

결과를 읽는 기준

구현 설명, 데이터셋별 실험 결과, 날짜가 있는 운영 관측을 구분하였다. 세부 조건과 남은 한계는 각 실험 옆에 제시하며, 목표나 기대 효과는 측정된 성과와 분리한다.

1 개발 배경과 요구사항

저장은 쉽지만 다시 찾고 활용하는 일은 여전히 사용자의 몫이었다.

스크린샷·북마크·메모는 여러 앱에 흩어지고, 저장한 이유와 후속 행동은 함께 남지 않는다. 모찌는 이 단절을 통합 저장, 의도 해석, 자연어 탐색, 능동 회수로 연결하는 것을 목표로 개발하였다.

기존 사용 방식	사용자가 추가로 해야 할 일	모찌의 접근
사진·스크린샷 보관	저장 이유를 기억하고 내용을 다시 읽기	이미지와 메모를 함께 해석
앱별 북마크	저장 위치와 제목을 기억하기	여러 입력을 모아 의미로 검색
메모·캘린더 수동 작성	분류·날짜·할 일을 다시 입력하기	용도를 분류해 일정·행동으로 연결

사용자 문제 확인

자체 조사 53명	응답 결과
조사 기간	2026년 6월 29일~7월 3일
저장한 기록을 다시 못 찾은 경험	42명 · 79.2%
저장한 스크린샷을 다시 보지 않은 경험	49명 · 92.5%

대상 사용자와 핵심 요구

- 과제·신청 공고를 저장하는 사용자: 마감과 해야 할 행동 확인
- 장소·제품·학습 자료를 모으는 사용자: 제목을 몰라도 관련 기록 탐색
- 기록을 쌓아 두고 잊는 사용자: 필요한 기록의 재노출과 후속 행동 지원

기능 요구는 수집·요약과 다중 분류·검색과 대화·기기 일정 연동·회수 알림이다. 공통 품질 요구는 소유권 분리, 중복 접수 방지, 민감정보 처리, 실패 후 재처리로 정리하였다.

2 사용자 경험과 서비스 기능

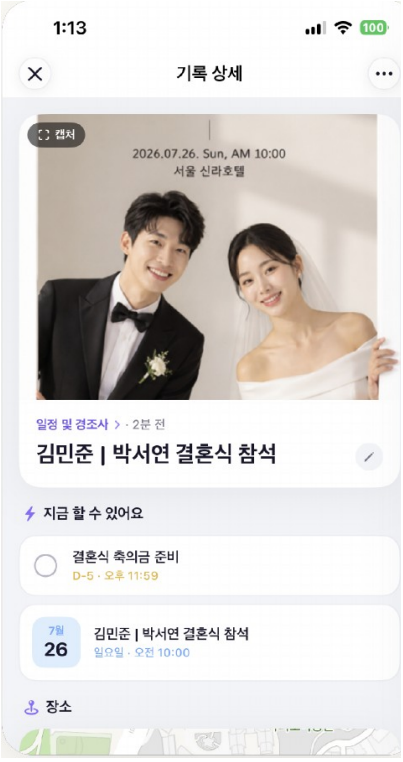
청첩장 한 장을 저장하면 참석 일정과 준비할 일이 함께 연결된다.



그림 1 저장한 정보가 다시 활용되는 전체 여정



① 메모와 함께 저장



② 요약과 행동 확인



③ 일정과 할 일 관리

청첩장 예시의 저장·정리·활용 장면.

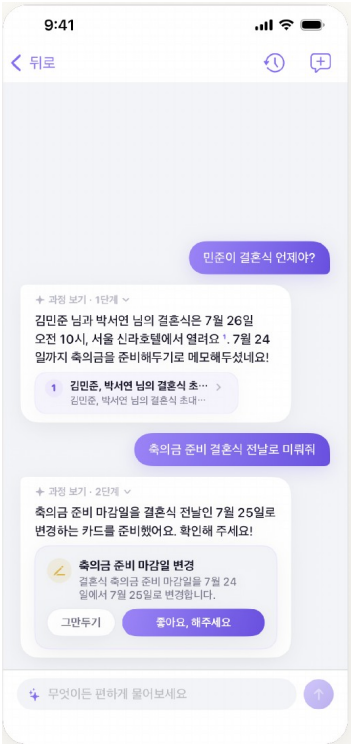
한 기록에서 여러 활용으로

"5일 뒤 축의금 준비"라는 메모와 청첩장을 함께 전달하면, 결혼식 참석 일정과 준비할 일을 분리해 관리할 수 있다. 사용자는 추출 결과를 확인하고, 기기 권한과 동기화 설정에 따라 캘린더·미리알림에 연결한다.

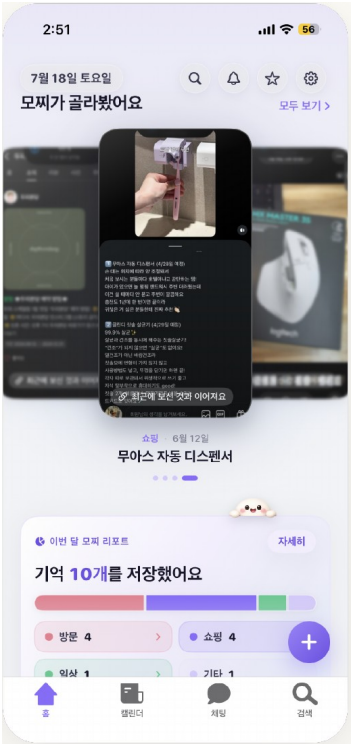
저장한 정보	분석 후 제공할 정보	사용자에게 남는 행동
청첩장 이미지·메모	요약·날짜·장소·의도별 항목	참석 일정 확인·준비할 일 관리

2 사용자 경험과 서비스 기능

필요할 때 찾고 잊기 전에 다시 보기



기록 근거를 함께 제시하는 대화



다시 볼 기록을 먼저 제시하는 홈

기능	사용자가 얻는 결과
통합 저장	공유 시트의 이미지·링크와 앱 내 텍스트를 한곳에 수집
AI 정리	요약·분류·핵심 정보와 불확실한 분류의 확인 후보
검색·질문	키워드·의미 검색 및 원본에 연결되는 답변
일정·할 일	날짜·마감과 완료 상태, 기기 캘린더·미리알림 연동
능동 회수	회수 피드·푸시 알림과 열람·무시·중지 제어

검색과 대화는 사용자가 필요를 인지했을 때의 재탐색을, 회수는 다시 볼 시점을 놓치기 쉬운 상황을 담당한다. 두 경로 모두 원본 기록에 연결해 내용을 다시 확인할 수 있도록 구성하였다.

3 시스템과 데이터 설계

기기 입력과 권한은 iOS가, 분석과 기록 처리는 FastAPI가 담당한다. 외부 AI·검색·스토리지는 어댑터로 연결하며, 관계형 데이터와 검색 벡터는 PostgreSQL·pgvector에서 관리한다.

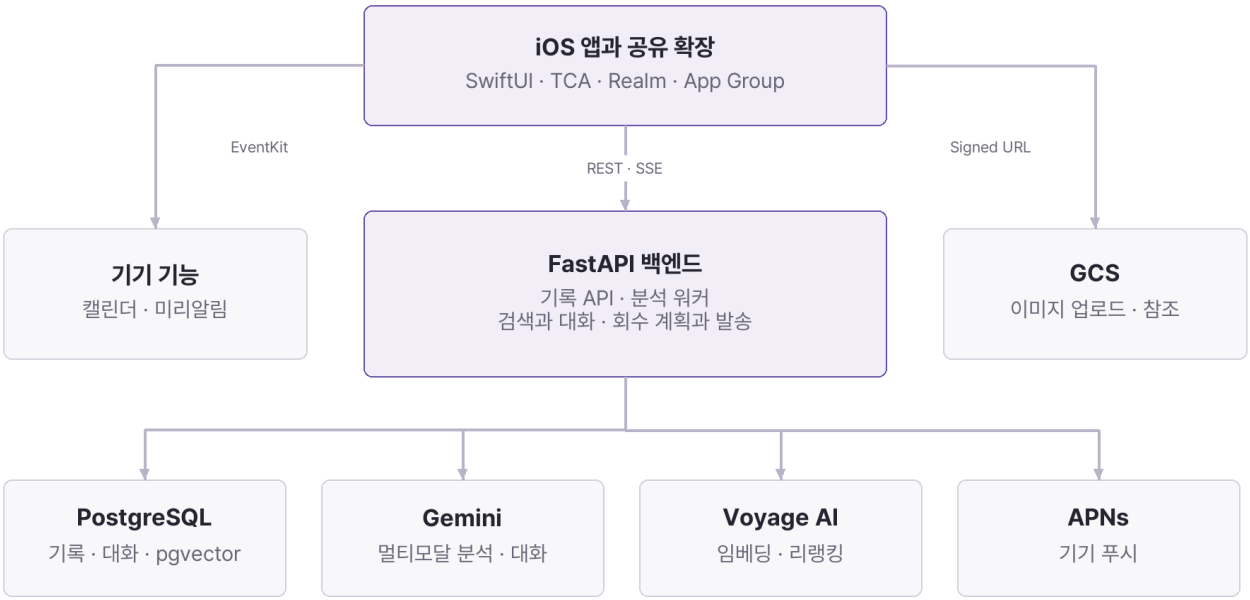


그림 2 서비스 논리 구성도 기능 구분과 데이터 연결

주요 기술과 역할

영역	기술	선택에 연결된 역할
iOS	SwiftUI·TCA·Tuist	상태·효과·기능 모듈 관리
로컬·기기	Realm·App Group·EventKit	공유 입력·로컬 데이터·기기 일정
API·DB	FastAPI·SQLAlchemy·Alembic	비동기 API·데이터 접근·마이그레이션
검색·AI	pgvector·Gemini·Voyage AI	의미 검색·멀티모달 분석·리랭킹
외부 연동	GCS·APNs·Apple·Google	이미지 업로드·푸시·로그인

분석·알림 워커는 공개 코드의 FastAPI lifespan에서 비동기 작업으로 기동한다. 논리적 모듈 분리를 독립 서버 배포와 구분하며, 운영 관측 시점의 배포 정보는 6장에서 제시한다.

3 시스템과 데이터 설계

원본 기록과 활용 단위를 분리한 데이터 모델

한 장의 기록에 여러 의도가 공존하므로 기록의 처리 상태와 행동의 생명주기를 분리하였다.

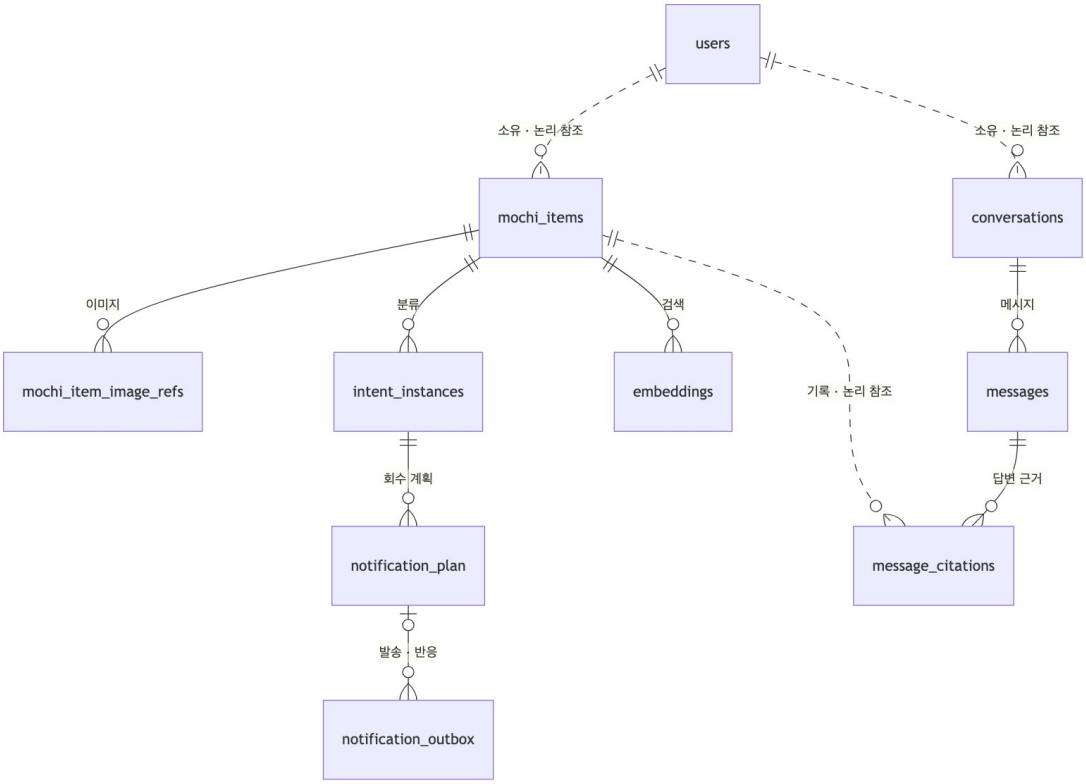


그림 3 주요 엔티티 관계 점선의 논리 참조와 DB 외래 키 구분

분리한 단위	데이터와 책임
원본 기록	mochi_items·이미지 참조·엔티티: 저장한 내용과 분석 상태
용도별 항목	intent_instances: 일정·할 일·지식·메모 및 개별 상태
검색·대화	embeddings·messages·message_citations: 탐색과 답변 근거
처리·회수	agent_runs·tool_calls·Outbox·notification_plan: 실행과 발송 이력

일정은 시간 도래, 할 일은 완료, 지식은 다시 보기와 오래됨, 메모는 보관과 수동 정리의 기준이 다르다. 답변의 인용은 제목·발췌를 스냅샷으로 보관하며, 소유자와 기록의 논리 참조는 조회·변경 시 검증한다.

3 시스템과 데이터 설계

복잡도를 늘릴 곳과 줄일 곳의 선택

기기 확장·상태 관리·여러 AI 기능을 지원하면서도 운영 단위를 과도하게 늘리지 않는 방향으로 구성하였다. 아래는 현재 구조와 대안의 비교이며, 실제로 실험한 AI·검색 선택은 5장에 구분해 제시한다.

설계 주제	선택과 이유	부담과 경계
앱 구조	SwiftUI·TCA와 기능 모듈: 상태와 비동기 효과 분리	초기 구조·의존성 학습 비용
서버 구조	Modular Monolith·Hexagonal: 도메인과 외부 구현 분리	독립 서비스 대비 운영은 단순하나 기능별 독립 확장은 제한
검색 저장소	PostgreSQL·pgvector: 관계와 벡터를 한 저장소에서 관리	별도 벡터 DB 대비 대상 감소, DB 부하·인덱스 관리 공유
분석 접수	DB Transactional Outbox: 기록과 작업의 원자적 접수	동기 분석 대비 접수·분석 분리, 폴링·잠금·재시도 필요
이미지 전달	Signed URL: GCS 직접 업로드	API 바이트 전달 분리, 서명·객체 소유권 검증 필요
의도 구조	원본 아래 의도 항목 1 대 다	단일 분류보다 표현력 증가, 항목별 상태 관리 필요

선택 이후의 관리

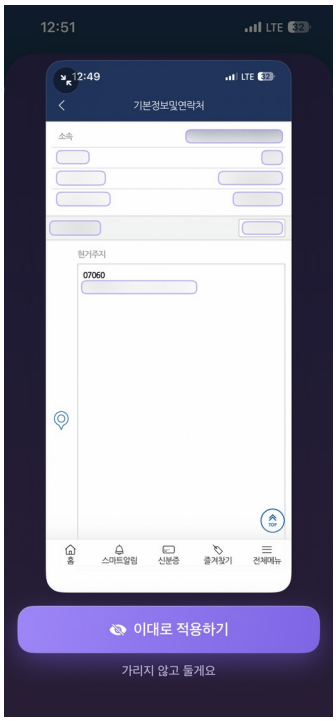
- 외부 모델·검색 어댑터 교체 시 품질 평가와 필요 시 재색인 수행
- 워커 처리량 증가 시 DB 연결 수와 외부 API 쿼터를 함께 검토
- 클라이언트와 서버의 상태 전이·API 계약을 같은 기준으로 유지

구조적 교체 가능성과 실제 품질의 무회귀는 별개이다. 모찌는 이 경계를 실험·정적 검사·모듈 경계 테스트로 관리하는 기반을 두었다.

4 핵심 기능 구현

입력 경로 통합과 업로드 전 보호

외부 입력은 Share Extension으로 합류한다. 백탭도 사용자 단축어의 스크린샷·공유 시트 경로를 사용하며, 앱 내 텍스트는 별도 화면에서 같은 기록 모델로 연결한다.





기기 내 Vision OCR과 감지 규칙으로 가림 영역 제시. 사용자의 업로드 정책에 따라 원본·가림본·전송 생략 경로 분리.

그림 4 기기 내 민감정보 가림 화면과 이미지 전달 흐름 팀 개발 폴더 화면 발췌

저장 중 유실과 중복을 함께 고려

순서	처리와 의도
먼저 남기기	App Group inbound 큐에 기록해 앱 활성화 후 다시 동기화
빠르게 전달하기	공유 확장에서 서버 직접 전송을 시도해 분석 조기 시작
같은 요청 구분하기	직접 전송과 큐 재전송에 같은 멱등키 사용
분석 보조 정보 정리	analysis_ocr_text를 임시로 사용하고 분석·실패 정리 후 제거

가림 탐지가 모든 민감정보를 찾는 것은 아니므로 확인·선택을 함께 제공한다. 로컬 Realm과 App Group은 앱·공유 확장의 입력을 연결하고, App → Features → Cores 방향으로 상태·저장·통신·기기 연동을 분리한다.

4 핵심 기능 구현

필요한 도구를 선택해 기록을 해석하는 분석

워커가 접수된 기록을 가져오면 에이전트는 시각·메모·기존 기록을 참고한다. 이미지 안의 정보가 부족할 때 링크 본문·플랫폼 메타데이터·검색 결과를 보조적으로 활용하고, 결과를 요약·의도·엔티티로 반영한다.

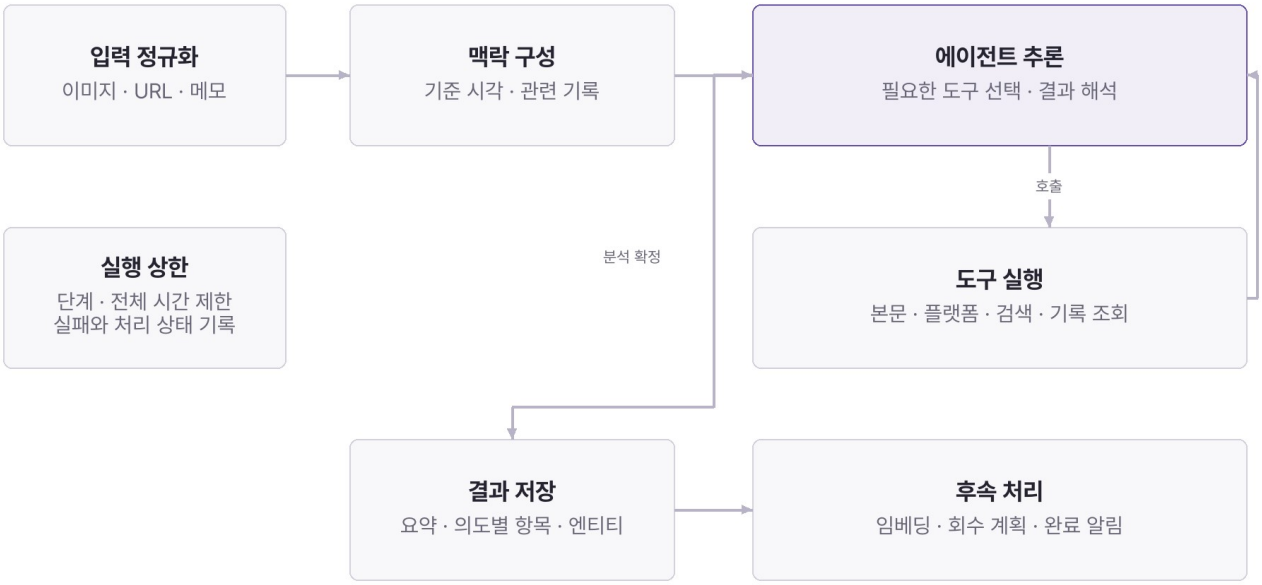


그림 5 분석 에이전트의 추론과 도구 실행 결과 저장과 후속 처리 분리

처리 지점	구현 의도
기준 시각과 문맥	상대 날짜·단문 메모를 입력 맥락과 함께 해석
도구 결과	외부에서 보충한 정보와 이미지의 직접 근거 구분
불확실한 분류	needs_review와 후보를 통해 사용자 확인
다중 의도	한 기록에서 일정과 준비할 일을 각각 생성
관측·상한	agent_runs·tool_calls와 단계·전체 시간 제한

코드 기본 상한은 최대 8단계·단계 30초·전체 분석 120초이다. 평균 지연이나 성능 목표가 아니라 실행 범위의 제한이며, 실제 관측값은 6장에서 별도로 다룬다.

4 핵심 기능 구현

근거로 답하고 확인 후 변경하는 대화

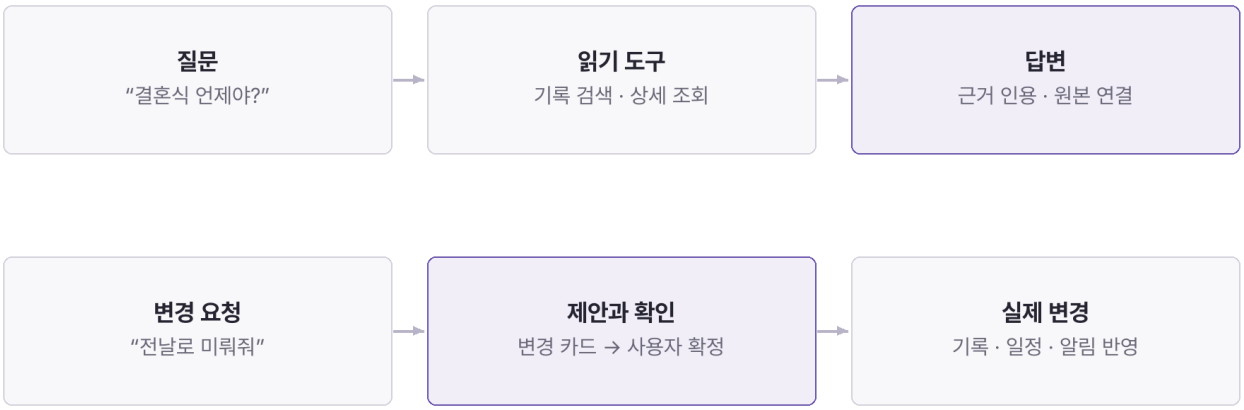


그림 6 읽기 도구와 사용자 확정 이후의 변경 경로

대화는 조회한 기록을 인용하고, 원본을 다시 열 수 있게 연결한다. "일정을 미뤄줘" 같은 쓰기 요청은 제안 카드로 구성해 확정 이후 변경한다. 답변·인용·제안 이벤트는 SSE로 전달하고 메시지는 서버 스냅샷으로 보관한다.

기록을 다시 떠올리게 하는 회수

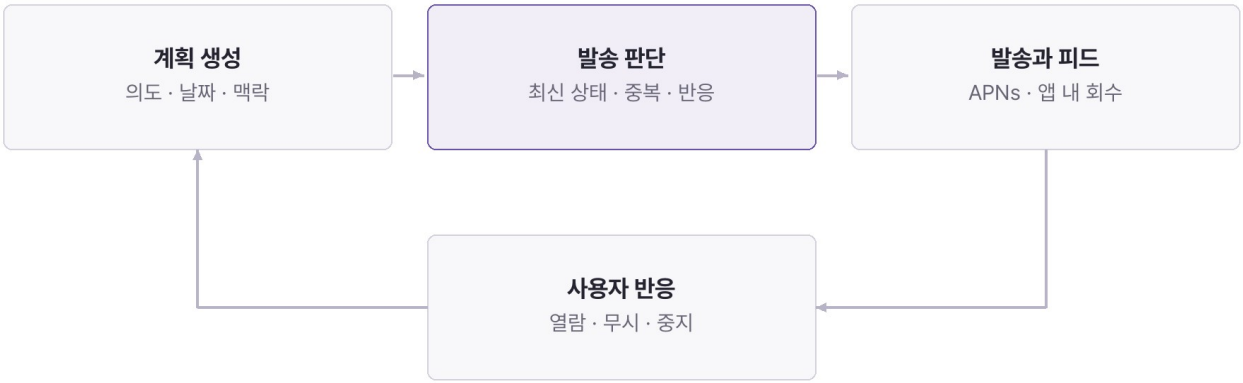


그림 7 계획 생성과 발송 판단을 나눈 회수 알림 반응의 후속 반영

notification_plan은 의도별 회수 계획을, notification_outbox는 발송과 반응을 관리한다. 계획을 만든 뒤에도 발송 시점의 최신 의도 상태와 중복·반응 조건을 확인하며, 앱 내 피드와 APNs 푸시로 연결한다.

4 핵심 기능 구현

계약과 실패 처리를 통한 기능 연결

대표 API 계약

메서드	경로 요약	역할
POST	/auth/apple · /auth/google	로그인
POST	/mochi_items/presign · /mochi_items	업로드 서명·기록 접수
GET	/mochi_items/{id} · /mochi_items/{id}/stream	상세·분석 상태
POST	/search	검색
POST	/conversations/{id}/messages	질문·SSE
PATCH	/conversations/{id}/proposals/{id}	제안 상태
GET·POST	/recall/feed · /recall/events/{id}/reaction	피드·반응

경로는 /api/v1 기준 요약이며 전체 파라미터·응답은 openapi.json 참조.

인증과 데이터 보호

- Apple·Google ID 토큰 검증 후 자체 access JWT·회전 refresh token으로 세션 관리
- 공유 확장 capture token은 기록 접수·서명에 제한된 권한으로 분리
- owner_id와 objectKey 소유권 검증, 로컬 개발 인증의 환경 제한
- 요청 ID와 공통 오류 응답으로 클라이언트·로그 연결

실패 후 다시 처리하기

상황	대응과 경계
중복 접수·워커 종료	소유자별 멱등키·영속 Outbox·잠금 만료 후 재청구
분석 반복 실패	최대 시도 후 dead_letter로 분리. 수동 복구·경보 검증 필요
외부 URL 접근	DNS·연결 대상 확인, 사설망·리다이렉트·MIME·크기 가드
대화 실행 중 프로세스 종료	메시지·run 상태 보관. 인메모리 실행의 복구는 별도 과제

분석 워커 기본값: 폴링 5초·잠금 180초·최대 5회. 재처리 기반이며 외부 부작용의 정확히 한 번 실행을 보장하는 의미는 아님.

5 문제 해결과 실험

사례 1 저장 의도를 놓치지 않는 분석

많이 추출하는 방식보다 용도에 맞게 구조화하고 검증하는 방식을 선택하였다.

화면과 짧은 메모에는 사실·개인 의견·해야 할 행동이 섞여 있다. 정보를 많이 나열해도 사용자가 왜 저장했는지 놓칠 수 있어 추출 형식과 모델을 함께 비교하였다.

비교 축	대안	관찰
A 자유형	자유로운 bullet 나열	간결하나 항목의 의미·타입 구분이 약함
B 구조화	facts·opinions·key_info·entities	세부 항목·출력 토큰 증가
C 타입 태그	fact·info·opinion 태그와 짧은 항목	타입 힌트와 간결함의 절충

동일 텍스트 40건의 비교

C 전략의 모델	의도 정답	건당 추정 비용
gemini-3.1-flash	35/40	약 $0.00051
gemini-3.1-flash-lite	37/40	약 $0.00014

flash-lite의 A·B·C는 모두 37/40으로, C가 정확도에서 독점적으로 우수한 것은 아니었다. A는 비용이 더 낮지만 타입 힌트가 적었고, B는 평균 추출 항목 4.7개로 C의 2.0개보다 장황했다. C와 flash-lite 조합을 간결성·구조·비용의 균형으로 채택하였다.

구현에 반영한 선택

- 타입 태그를 이용한 추출 기준을 분석 프롬프트에 반영
- 원본 기록 아래 여러 의도 항목을 두어 일정과 준비할 일을 함께 관리
- 확정하기 어려운 분류는 사용자 확인 대상으로 분리

조건: 2026년 5월 29일, 스크린샷 22장·텍스트 40건. 정답 기반 의도 비교는 텍스트 40건에 한정. 이미지에는 해당 gold 정확도 없음. 비용은 당시 코드 단가 추정치이며 현재 요금과 구분.

5 문제 해결과 실험 사례 1

분류 기준을 보완하고 회귀로 확인

v28에서 v29로 전체 run 수준 점수 91.3% → 95.6%

이후 골든 평가를 229케이스로 확장하고 케이스당 3회 실행하여, 경계 분류·날짜·표 분해를 따로 점검하였다. 실패 클래스에 맞춰 프롬프트 기준을 보완하고 같은 조건에서 다시 비교하였다.

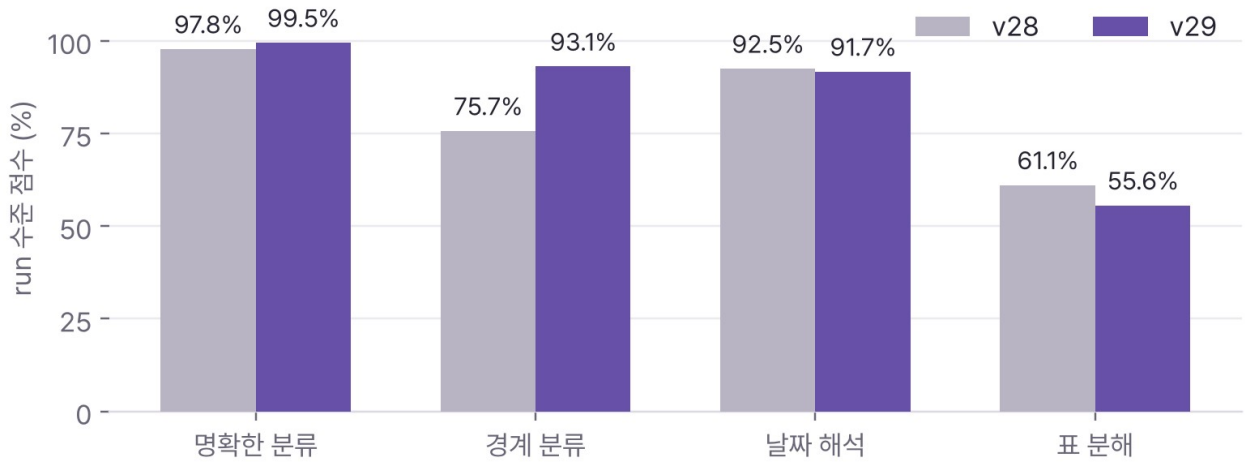


그림 8 v28과 v29의 스위트별 run 수준 점수 동일 평가 기록

평가 단위	케이스 수	v29 결과
명확한 분류	135	99.5%
경계 분류	48	93.1%
날짜 해석	40	91.7%
표 분해	6	55.6%
전체	229 · 총 687 run	657/687 · 95.6%

개선과 남은 실패

경계 분류는 75.7%에서 93.1%로 개선되었다. 반면 날짜와 표 분해는 낮아져, 전체 점수 상승이 모든 입력의 개선을 뜻하지는 않았다. 공휴일명 날짜 해석과 작은 표 분해 스위트는 후속 보완 대상으로 남겼다.

2026년 7월 8일 기록. 전체 Wilson 95% 구간 93.8~96.9%는 run 수준 기록값. 같은 케이스의 반복은 비독립이며 케이스 단위 엄격 판정도 필요. gold 라벨 팀 리뷰 전제가 있어 일반 스크린샷 정확도로 전용하지 않음.

5 문제 해결과 실험

사례 2 기억나는 말로 원하는 기록 찾기

검색 문서와 모델을 고른 뒤 후보 회수와 상위 순위 개선을 나누어 검증하였다.

원문에 없는 표현으로도 찾아야 하고, 비슷한 기록이 많아도 정답이 앞에 나와야 했다. 임베딩 입력·차원을 먼저 비교하고, 별도 데이터에서 리랭커의 효과를 검증하였다.

모델 선정 실험	비교와 선택
조건	46문서·30 gold 질의·45개 조합 · 2026년 6월 9일
선택	voyage-4-large 1024차원 × summary+labels
결과	nDCG@5 0.925 · HitRate@3 1.000 · MRR 0.944

상위 순위는 리랭커로 개선

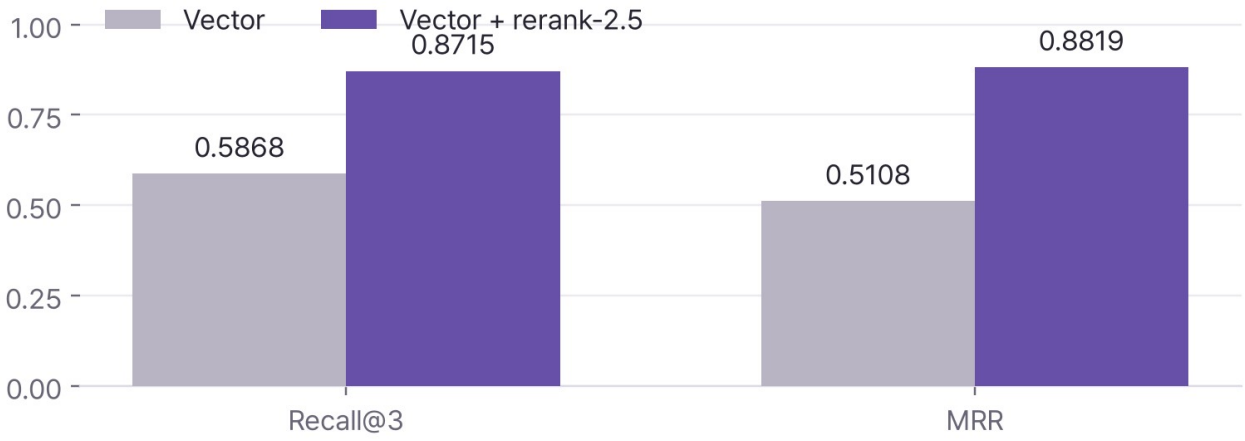


그림 9 391문서·24질의에서의 리랭커 비교 7월 9일 정정 재측정

Vector 단독의 Recall@3 0.5868·MRR 0.5108이 rerank-2.5 결합 후 0.8715·0.8819로 상승하였다. 같은 실험에서 Hybrid+reranker는 Vector+reranker와 동일해, 키워드 결합 자체의 추가 회수 이득은 관측되지 않았다.

구현과의 연결

검색탭은 키워드·벡터 후보의 RRF 결합과 리랭킹 경로를 유지한다. 키워드 검색은 단어별 ILIKE와 PostgreSQL 순위 함수 기반 근사이며, 실험용 BM25와 동일 알고리즘으로 취급하지 않는다. 임베딩 불가 시 키워드 폴백을 제공한다.

46문서 모델 선정과 391문서 리랭커 비교는 독립 실험. 초기 voyage-3-large 값과 정정된 voyage-4-large 값을 섞지 않았으며, 위 그래프는 정정 재측정만 사용.

5 문제 해결과 실험 사례 2

다정답 질의에서 반환 수의 한계 확인

상위 순위만 좋아져도 여러 정답을 모두 담지 못하면 “저장한 전시 정보 전부” 같은 질의는 실패한다. 작은 실험의 Recall@10 천장 효과를 보완하기 위해 1,045개 아이템·50개 질의의 증강 코퍼스에서 K를 비교하였다.

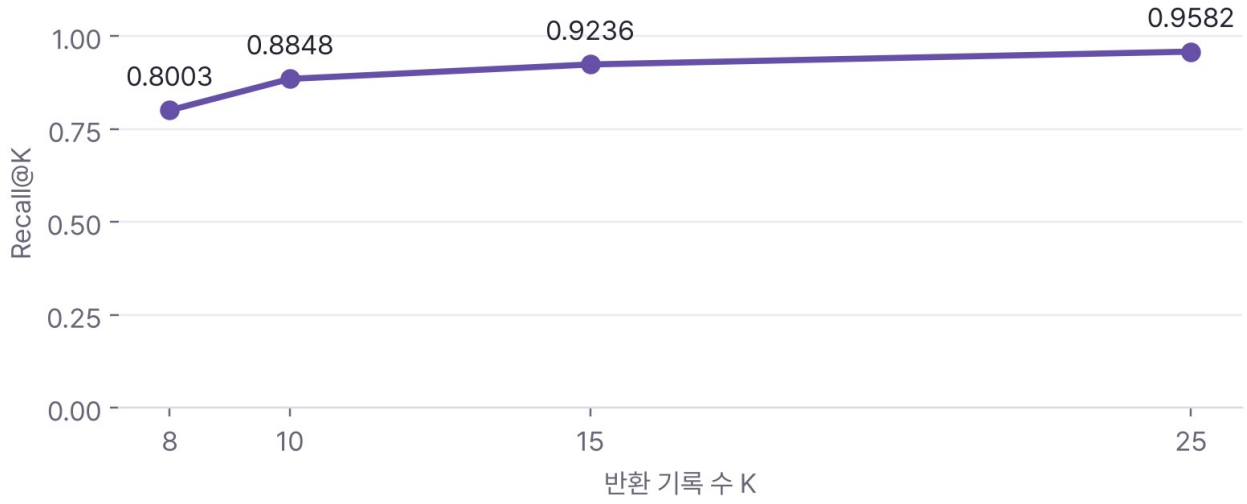


그림 10 채팅 형상 C의 감사 라벨 기준 Recall@K 반환 수에 따른 회수 범위

문제와 선택

관찰	선택에 연결된 의미
감사 라벨 전체 0.8003 → 0.9582	K 8에서 25로 늘려 다정답 누락 축소
다정답 L군 0.627 → 0.896	정답이 8개보다 많으면 슬롯 부족이 구조적 제약
후보 recall이 상한에 근접	다음 개선은 K만이 아니라 후보 폭·질의 분포 검토

라벨 보정 후에도 같은 방향

라벨 기준	Recall@8	Recall@25
독립 감사	0.8003	0.9582
retrieval pool 보정	0.8069	0.9661
pool 보정과 중복 그룹	0.8069	0.9718

K=25의 근거는 순위 자체의 정확도 상승보다 회수할 수 있는 정답의 수를 늘린 데 있다. 문맥 토큰과 지연 비용을 함께 보아야 하며, 단일 정답 정밀도를 다룬 리랭커 실험과 결론을 합치지 않는다.

2026년 7월 14일. 합성·증강 코퍼스와 질의별 gold 기준의 회수율이며, 전체 실사용 검색 정확도로 일반화하지 않음.

5 문제 해결과 실험

사례 3 필요한 기록을 적절한 때 다시 알리기

각 기록의 계획을 독립적으로 만들면 같은 날 알림이 몰릴 수 있었다.

저장 시각이 곧 알림을 반길 시각이라는 가정을 두지 않고, 사용자 기록과 계획 결과로 확인하였다. 약한 프롬프트의 독립 호출 A와 강한 규칙·시간순 이전 결정 맥락을 함께 제공한 B를 비교하였다.

비교군	입력과 제약
A 독립 계획	각 기록을 약한 프롬프트로 독립 처리
B 조율 계획	강한 규칙과 시간순 이전 계획을 함께 제공

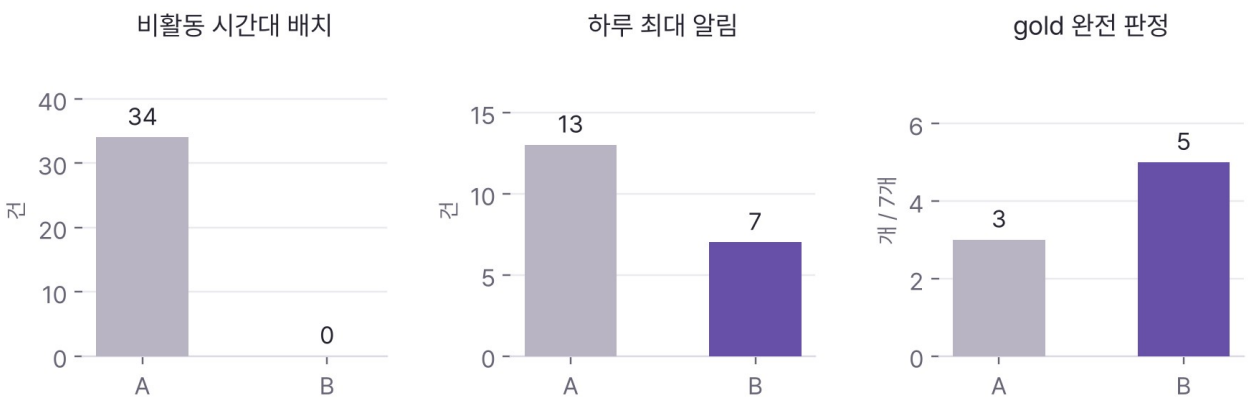


그림 11 38케이스·152회 호출의 플래너 비교 실험 사용자·설정에 한정

선택과 구현

B에서는 실험 사용자의 비활동 시간대 배치가 34건에서 0건으로, 하루 최대 알림이 13건에서 7건으로 줄었다. gold 7개 중 완전 판정도 3개에서 5개로 늘어, 계획을 함께 조율하는 방향의 근거로 삼았다.

모델은 계획을 제안하고 실행 계층은 상태·중복·반응을 확인하도록 역할을 분리하였다. 발송 정책은 현재 코드·프롬프트·배포 설정에 따라 달라지므로 과거 실험의 시간대나 횟수를 고정 운영 정책으로 일반화하지 않는다.

안정성 해석의 교정

순서 셔플 exact는 A 1.0·B 0.053이지만 B는 입력 맥락 자체가 바뀐다. 이 차이를 그대로 모델 비결정성으로 해석하지 않고, 시간순 조율의 이득과 별도로 남겼다.

5 문제 해결과 실험 사례 3

묶기와 이름 재사용의 이득과 비용

관련 기록을 한 주제로 묶고 같은 카테고리 이름을 재사용하면 재탐색과 회수의 일관성이 높아질 수 있다. 반대로 다른 기록까지 합쳐지는 오류가 생길 수 있어 단일 유사도 임계값과 맥락 기반 판단을 비교하였다.

주제 묶기

비교와 관찰	결정
단일 cosine 임계값에서 한 군집이 93건 흡수	유사도 하나로 최종 주제를 확정하는 방식 기각
LLM 6개 주제 중 5개 purity 1.0·1개 0.83	임베딩 후보 선별과 LLM 할당 결합 방향 채택
시간이 떨어진 같은 주제의 이름 중복	라벨 후보 소환을 통한 증분 재사용 검토

103개 아이템의 제한된 실험. 주제 수와 purity를 전체 사용자에게 보장하는 값으로 해석하지 않음. [1][9]

카테고리 이름의 일관성

관측 지표	이전	변경 후
이름 재사용 exact	0/24	24/24
false reuse	0/16	4/16

발표자료의 별도 live A/B 25케이스·100회 호출에서는 이름 일관성이 개선되는 대신 잘못 재사용하는 경우도 늘었다. 일관성만 최대화하지 않고 새 주제를 보존할 수 있는 기준과 수정 경로를 함께 고려하였다.

이 사례가 보여 준 개선 방식

- 불편한 결과도 유지: 군집 쏠림과 false reuse를 기각·보완의 근거로 기록
- 단일 점수보다 역할 분리: 임베딩은 후보, 모델은 맥락 판단에 활용
- 계획 품질과 실제 효과 구분: 알림 열람·피로도·행동 완료는 별도 검증

6 검증 체계와 운영 관측

구현 규칙과 모델 품질, 실제 운영 상태는 서로 다른 방법으로 검증하였다. 요구사항별 확인 지점을 연결하여 기능이 있다는 설명과 동작을 입증하는 근거를 구분한다.

요구사항	검증 지점	확인 수단
입력·중복 방지	멱등 저장·큐와 직접 전송·소유권	ingest·object ownership 테스트
분석·임시 정보	의도 상태·OCR 정리·프롬프트 회귀	lifecycle·ephemeral OCR·golden eval
검색·대화	후보·순위·인용·제안 상태	retrieval·RRF·chat run 테스트
알림·반응	발송·예약·열람·중지 상태	recall 계획·반응·worker 테스트
모듈 구조	계층과 import 방향	모듈 경계 테스트·정적 검사
기기 경험	권한·공유·캘린더·알림 수신	실기기 시나리오·권한 조합 확인

모델 평가의 재현 단위

- 데이터·gold·모델·프롬프트·입력 전략·지표를 실험별로 고정
- 정답과 평가 대상의 역할 분리, 날짜·라벨 오류도 함께 검토
- 같은 케이스 반복 실행을 독립 사용자 표본으로 과대 해석하지 않음
- 라벨 보정 전후와 채택·기각 결과를 함께 보존

개발 시 실행 가능한 검사

영역	명령과 목적
백엔드	make test · make lint · make typecheck
iOS	tuist test · SwiftLint·TCA TestStore 등
로컬 실행	uv·Docker Compose·Alembic / Xcode·Tuist

파일과 검사 명령의 존재는 최근 통과 증빙과 구분. 이 개정에서는 앱·서버 전체 테스트를 새로 실행하지 않았으며, 실제 CI 실행 여부는 실행 이력 기준으로 확인.

6 검증 체계와 운영 관측

운영 기록으로 본 응답과 데이터 완결성

작동 여부를 넘어 누가 무엇을 기다렸고 어떤 데이터가 빠졌는지 확인하였다.

2026년 10월 20일 13시 15분 UTC의 저장 집계는 배포본 449c49a·Alembic 0028 기준이다. 계정 15개에는 데모 페르소나 3개와 팀원이 포함되므로 외부 사용자만의 성과로 제시하지 않는다.

관측 항목	결과	분모와 의미
분석된 live 기록	634건	삭제 제외·분석 상태 기준
채팅 누적 상태	완료 541·실패 5턴	누적 상태 분포
완료 채팅 전체	p50 5.73초 / p95 12.86초	완료 541턴·실패 제외
첫 툴 완료	p50 2.77초 / p95 6.93초	툴 사용 515턴·첫 토큰 아님
캡처 → 첫 성공 분석	p50 12.96초 / p95 26.02초	첫 성공 699아이템·푸시 수신 전

실패 로그가 없어도 색인이 빠질 수 있음

미색인 8건의 구분	관측과 해석
7건	요약·라벨이 비어 색인할 문서가 없는 기록
1건	confirmed_instances 우회 경로로 색인 단계 미실행
파이프라인 경유 565건	해당 집계에서 임베딩 누락 0건

8건 전체를 임베딩 호출 장애로 세지 않고 원인별로 구분하였다. 실패 재시도만으로 잡히지 않는 우회 경로가 있어, 상태 비교형 보정과 생성 경로별 색인 계약이 후속 과제로 드러났다.

완료 턴 백분위는 타임아웃 실패를 제외하며 분석 완료는 푸시 수신 시각과 다름. 최근 30일 채팅 사용자는 1명으로 장기 효과의 근거는 제한적. 로컬 실험 개선치를 이 운영 지연의 개선폭으로 연결하지 않음.

7 구현 성과와 향후 개선

분석·검색·대화·회수를 하나의 기록 모델로 연결하고 선택의 근거를 남겼다.

성과	프로젝트에서 확인 가능한 내용
서비스 연결	공유 입력 → 분석 → 검색·대화 → 일정·회수까지 iOS·API로 연결
분석 개선	추출 형식·모델 비교와 프롬프트 회귀 평가로 선택을 구체화
검색 검증	모델 선정·리랭킹·K 확장을 다른 질문과 데이터로 검증
알림 조율	강한 규칙·맥락 결합의 이득과 순서 해석의 한계 확인
개인 기록 보호	기기 내 가림·소유권 필터·근거 인용·변경 제안 확인
외부 성과	2026 SW중심대학 디지털 경진대회 SW부문 최우수상

사용자 가치로 연결되는 지점

사용자가 화면을 다시 읽고 여러 앱에 옮겨 적는 수고를 줄이는 것이 구현의 지향점이다. 검색 결과에 근거를 남기고 일정·회수로 연결하는 경험은 단순 저장량보다 기록이 다시 활용되는지를 평가하도록 만든다.

실제 효과를 확인할 지표

평가 질문	후속 측정 방법
정리 부담이 줄었는가	동일 과제에서 수동 입력·정리 시간 비교
원하는 기록을 찾았는가	사용자 과제별 재탐색 성공률·소요시간
알림이 도움이 되었는가	열람·무시·중지와 행동 완료를 분리 집계
계속 다시 쓰는가	외부 사용자 코호트의 기록 재사용·유지 추적

위 사용자 효과는 기대 가치와 후속 검증 계획이며 달성 수치가 아님. 수상은 사용자 제공 사실에 따른 표기.

7 구현 성과와 향후 개선

실패 유형에서 다음 개발 우선순위로

영역	남은 한계	개선과 완료 판단
분석	공휴일명 날짜·표 분해·경계 분류	실패 gold 보강·보수적 확인·기존 스위트 회귀
검색	합성셋과 후보 천장·실제 질의 분포 차이	고유명사·다정답 실질의 평가와 후보 단계 진단
회수	소규모 사용자·시간대와 맥락의 편향	열람·피로도·행동 결과를 사용자별로 검증
데이터 완결성	분석 우회 경로의 색인 누락	생성 경로별 색인 확인과 상태 비교형 보정
외부 연동	URL 복원·API 응답·기기 권한 의존	폴백·상태 안내와 권한별 종단 시나리오
운영 효과	팀원·데모 포함과 낮은 최근 사용량	외부 코호트 분리·충분한 기간의 추적

우선순위별 개선 방향

단계	우선 작업	확인할 결과
1 정확성	날짜·분류·누락 색인 등 재활용을 막는 오류 보완	실패 사례 해소와 회귀 유지
2 실제 활용	검색 성공과 알림 이후 행동의 사용자 평가	과제 성공·부담·피로도 확인
3 확장	사용량·부하를 바탕으로 워커·캐시·플랫폼 확대 검토	품질·비용·운영 복잡도 비교

특정 기술을 추가하는 것보다 현재 기록을 믿고 다시 사용할 수 있는지를 먼저 확인한다. 확장 선택은 실제 처리량·실패 분포·비용에 근거해 판단하며, 아직 측정하지 않은 유지율이나 사업 추정을 구현 성과로 제시하지 않는다.

8 팀과 프로젝트 산출물

이름	학과	학번	역할
송준서	소프트웨어학과	32202337	팀장
박세현	소프트웨어학과	32211690	팀원
이용찬	소프트웨어학과	32213336	팀원

지도교수 이성한 교수 · 제출일 2026년 11월 16일

개발 기록의 주요 시점

시점	확인된 기록
2026년 6월 1일	미션·4가지 의도·복합 용도와 개발 계획 논의
6월 9~16일	검색 모델·리랭커·알림 계획 비교
7월 8~14일	확장 골든 평가·모델 정정·TOP_K 평가
10월 20일	운영 DB·로그 집계
11월	캡스톤 README·결과보고서 정리

제출 산출물

산출물	위치
소스코드	https://github.com/dku-mochi/mochi
시연 영상	https://www.youtube.com/watch?v=Y2UenlJ6z1w&t=2s
발표자료·최종 발표자료	docs/mochi-presentation.pdf · 동일 파일
API 명세	mochi-backend/openapi.json
개발·실행 안내	mochi-ios/README.md · mochi-backend/README.md
결과보고서	모찌 프로젝트 결과보고서 · 본 문서

9 참고자료

[1] 세얼간이 발표자료 사용자 제공 58쪽 PDF. 문제 조사·서비스 화면·실험 요약.

[2] 공개 구현과 API github.com/dku-mochi/mochi · main 43f036b 기준. openapi.json·app/modules·Targets·tests.

[3] 추출 전략 비교 lab/archive/extraction-strategy/FINDINGS.md 및 extraction_strategy_text_report.md · 2026년 5월 29일. 이미지 22장·텍스트 40건.

[4] 골든 평가 확장 mochi-backend/eval/REPORT.md · 2026년 7월 8일. v28·v29, 229케이스·687 run 및 라벨·판정 한계.

[5] 임베딩 모델 선정 lab/archive/rag-search/FINDINGS.md · 2026년 6월 9일 최종 결과·6월 10일 차원 비교. 46문서·30질의.

[6] 리랭커 비교 정정 lab/archive/rag-rerank-eval/FINDINGS.md · 2026년 7월 9일 모델 정정 재실행. 391문서·24질의.

[7] 반환 수 비교 lab/active/rag-topk-eval/FINDINGS.md · 2026년 7월 14일. 1,045아이템·50질의와 감사·pool 보정.

[8] 회수 플래너 실험 journal/2026-06-16-proactive-recall-notification-planner.md · 실험·설계 로그. 당시 실험과 현재 구현 구분.

[9] 묶기와 카테고리 lab/proactive-recall-bundling/RESULTS.md 및 발표자료 44쪽. 103개 아이템의 묶기와 별도 카테고리 A/B 요약.

[10] 운영 집계 lab/active/prod-aggregates/RESPONSE.md·FINDINGS.md · 2026년 10월 20일 13시 15분 UTC. 배포본 449c49a·Alembic 0028.

[11] 초기 팀 회의 docs/meetings/2026-06-01.md · 미션·초기 결정·개발 계획. 최종 개인별 기여와 구분.

실험·운영 문서 경로는 팀 개발 폴더 기준. 공개 코드의 현재 기능과 과거 측정 조건을 분리하여 참조.